Part 2: Technical Details and Architectures of Streaming Systems

In this train, we'll be exploring streaming technology and its transformative impact on big data analytics. We will be covering the fundamental concepts of streaming, its necessity in the modern data landscape, and how it revolutionizes data processing and analysis.

Learning Objectives

By the end of this train you should be able to:

- Explain the difference between batch processing and stream processing.
- Understand the need for streaming.
- Understand the common architectures around which streaming tools are built.
- Identify the challenges associated with creating streaming applications.
- Describe and illustrate common big data architectures which optimally leverage streaming technologies.

Batch Vs Stream Processing

Batch Processing

Batch processing collects and processes data in fixed-size chunks at scheduled intervals. It’s suitable for use cases where real-time feedback isn't critical—like payroll or monthly reports.

Examples: Hadoop MapReduce, traditional ETL pipelines.

Stream Processing

Stream processing deals with continuous data inputs in real-time. It’s ideal for applications requiring instant decision-making like fraud detection or live analytics.

Examples: Apache Kafka, Apache Flink, Spark Structured Streaming.

Why Is Streaming Necessary?

In a world of real-time applications, static batch processes often fall short. Businesses now require instant feedback and responsiveness to stay competitive. Streaming enables this with:

- realtime analytics and insights.
- Faster incident detection (e.g. fraud, system errors).
- Improved user experiences (e.g. live recommendations).

Common Streaming Architectures

There are three broad categories of architectures used in stream processing:

- **Lambda Architecture:** Combines batch and stream processing for accuracy and speed. Data is processed via both a batch layer and a real-time layer.
- **Kappa Architecture:** Simplifies Lambda by using only stream processing. It reprocesses streams if necessary for recomputation.
- **Microservices-based Architecture:** Each processing step is a lightweight service, enabling modular design and better scalability.

Tools like Apache Kafka, Apache Flink, Apache Spark Streaming, and AWS Kinesis are commonly used in these setups.

Challenges of Streaming Applications

- **Data Ordering:** Ensuring data arrives and is processed in the correct sequence.
- **Fault Tolerance:** Recovering from system failures without data loss.
- **Latency:** Balancing performance with near real-time response.
- **Scalability:** Handling unpredictable volumes of data efficiently.

- **State Management:** Preserving context during stream computation (e.g. counts, averages).

Example: Real-world Streaming Architecture

Consider a real-time fraud detection system:

- **Kafka** ingests transaction data as it happens.
- **Flink** processes each transaction stream and applies fraud-detection logic.
- **Redis** stores stateful data (e.g. running totals per user).
- **Alerts** are triggered if suspicious patterns are detected and sent to an alerting service or dashboard in **Grafana** or **Power BI**.

Conclusion

Streaming systems represent a major evolution in data processing. From IoT sensors to stock trades, the ability to process data instantly unlocks powerful capabilities. Understanding the architectures behind streaming helps you build robust, scalable, and real-time applications for modern data demands.

Key Takeaways

- **Takeaway 1:** Batch processing handles data in bulk; stream processing does it in real time.
- **Takeaway 2:** Streaming is necessary for real-time insights and responsiveness.
- **Takeaway 3:** Lambda and Kappa are the two most common streaming architectures.
- **Takeaway 4:** Challenges in streaming include data ordering, fault tolerance, and low latency.
- **Takeaway 5:** Effective streaming systems rely on the right tools and architectural choices to meet business needs.